

Universidad Mariano Gálvez de Guatemala.

Campus de Huehuetenango.

Ingeniería en Sistemas de Información y

Ciencias de la Computación.

Desarrollo Web y Análisis de Sistemas II

Ing. Gilberto Martínez | Ing. Jorge García



Manual Técnico

Marcos Alejandro Molina Rivera | Carné: 22-3232

Mario Noel Molina Che | Carné: 22-2181

Sección: "A"

28 de octubre del 2025

Anexo A: Manual Técnico del Proyecto

Este documento consolida la documentación técnica para el **Backend (NestJS)** y el **Frontend (React)** del sistema Cine-App. Sirve como un manual de instalación, configuración y referencia de la API para desarrolladores.

A.1 Backend (NestJS API + Prisma)

A.1.1 Descripción General

Este proyecto corresponde al **backend** de un sistema de cine, desarrollado con **NestJS**, **Prisma** y **PostgreSQL**. Se ha implementado un esquema modular y en capas, siguiendo las buenas prácticas de NestJS, utilizando **DTOs**, **Services**, **Controllers**, **Modules**, **Guards**, **Decorators** y **Auth con JWT + bcrypt**.

A.1.2 Tecnologías Utilizadas

- [NestJS](#) - Framework para Node.js con arquitectura modular.
- [TypeScript](#) - Lenguaje base para mayor tipado y escalabilidad.
- [Prisma](#) - ORM moderno para comunicación con la base de datos.
- [PostgreSQL](#) - Motor de base de datos relacional.
- [JWT](#) - Manejo de autenticación y sesiones.
- [bcrypt](#) - Encriptación de contraseñas.
- [Swagger \(OpenAPI\)](#) - Documentación de API autogenerada.
- [Multer](#) - Middleware para la subida de archivos (pósters).

A.1.3 Estructura del Proyecto

El proyecto sigue la filosofía de **arquitectura modular en capas**:

Cada módulo cuenta con su propia carpeta que incluye:

- **Controller** → Maneja las rutas y peticiones HTTP. (Decorado para Swagger).
- **Service** → Contiene la lógica de negocio.
- **DTOs** → Validación y tipado de datos (`class-validator`).

- **Module** → Ensambla el módulo dentro del ecosistema NestJS.

A.1.4 Autenticación

Se utiliza **JWT** para la gestión de sesiones y roles.

- Las contraseñas se almacenan usando **bcrypt** (hash + salt).
- Se incluyen **Guards** (JwtAuthGuard) y **Decorators** (@Roles) para proteger rutas y manejar permisos según roles (ej. 'Admin', 'Casher').

A.1.5 Instalación y Configuración del Backend

1. Clonar el repositorio

```
git clone https://github.com/Mariooooooc/Backend-Cine.git
cd Backend-Cine
```

2. Instalar dependencias

```
npm install
```

3. Configurar variables de entorno

Crear un archivo .env en la raíz del proyecto con el siguiente contenido:

Fragmento de código

```
# URL de conexión a la base de datos
DATABASE_URL="postgresql://
usuario:password@localhost:5432/cine_db"

# Secreto para firmar los tokens JWT
JWT_SECRET="TU_PALABRA_SECRETA_PARA_JWT"

# Puerto del servidor
PORT=3000
```

4. Ejecutar migraciones con Prisma

```
npx prisma migrate dev
```

5. Levantar el servidor

```
npm run start:dev
```

Scripts disponibles:

- `npm run start` → Inicia la app en modo producción.
- `npm run start:dev` → Inicia en modo desarrollo con hot reload.
- `npm run build` → Compila el proyecto a JavaScript.
- `npx prisma studio` → Interfaz visual de Prisma para la BD.

A.1.6 Documentación de la API (Endpoints)

El backend autogenera una documentación interactiva de Swagger. Una vez que el servidor está corriendo (`npm run start:dev`), se puede acceder a ella desde:

<http://localhost:3000/documentation>

A continuación, se listan los endpoints principales del sistema:

Módulos de Autenticación y Usuarios

- **Auth (/auth):** Endpoints para login, register, refresh-token.
- **User (/user):** CRUD para la gestión de usuarios.
- **Role (/role):** CRUD para la gestión de roles (ej. "Admin", "User").
- **Profile (/profile):** Endpoint protegido (GET) que devuelve el perfil del usuario autenticado.

Módulos de Administración (Rutas Protegidas)

- **Admin (/admin):**
 - GET /admin/dashboard: Dashboard principal (Roles: 'Admin', 'Casher').
 - POST /admin/sales: Registro de ventas (Roles: 'Admin', 'Casher Food').
- **Employee (/employee):** CRUD para gestionar la relación entre un Usuario y un Empleado.

- **Supplier (/supplier):** CRUD para la gestión de proveedores.

Módulos de Películas (Catálogo)

- **Movie (/movie):**
 - GET /movie: Obtiene todas las películas.
 - GET /movie/:id: Obtiene una película por ID.
 - POST /movie: Crea una nueva película (subiendo un póster).
 - **Tipo:** multipart/form-data
 - **Campo 1 (file):** file (Archivo de imagen)
 - **Campo 2 (string):** data (JSON con CreateMovieDto)
 - PUT /movie/:id: Actualiza una película.
 - DELETE /movie/:id: Elimina una película.
- **Director (/director):** CRUD para directores.
- **Genre (/genre):** CRUD para géneros (ej. "Acción", "Drama").
- **AgeRating (/age-rating):** CRUD para clasificaciones de edad.

Módulos de Salas y Asientos

- **Room (/room):** CRUD para salas de cine.
 - POST /room/assign-seats: Endpoint especial para asignar asientos a una sala.
- **Seat (/seat):** CRUD para el catálogo maestro de asientos (ej. "A1", "A2", "B1").

Módulos de Productos (Dulcería/Tienda)

- **Categorie (/categorie):** CRUD para categorías de productos (ej. "Bebidas", "Palomitas").
- **Size (/size):** CRUD para tamaños de productos (ej. "Pequeño", "Grande").

A.2 Frontend (React + TypeScript + Vite)

A.2.1 Descripción General

Este proyecto corresponde al **frontend** del sistema Cine-App. Es una Aplicación de Página Única (SPA) construida con **React** y **Vite**, utilizando **TypeScript** para un tipado estricto y **TailwindCSS** para el diseño de la interfaz de usuario.

A.2.2 Tecnologías Utilizadas

- [React](#) - Biblioteca principal para la construcción de interfaces.
- [Vite](#) - Herramienta de construcción y servidor de desarrollo rápido.
- [TypeScript](#) - Lenguaje base.
- [TailwindCSS](#) - Framework de CSS utility-first.
- [React Router DOM](#) - Para el manejo de rutas del lado del cliente.
- [Axios](#) - Cliente HTTP para la comunicación con el Backend.
- [Framer Motion](#) - Para las animaciones de la interfaz.
- [Lucide React](#) - Biblioteca de íconos.

A.2.3 Instalación y Configuración del Frontend

1. **Navegar a la carpeta del frontend** (*Asumiendo que está en una subcarpeta del repositorio principal*)

```
cd ../Frontend-Cine
```

2. **Instalar dependencias**

```
npm install
```

3. **Ejecutar el servidor de desarrollo**

```
npm run dev
```

La aplicación estará disponible en `http://localhost:5173` (o el puerto que indique Vite).

A.3 Autor del Proyecto

- Desarrollado del Backend: **Mario Noel Molina Che**

- Colaborador del Frontend: **Marcos Alejandro Molina Rivera**
- Proyecto académico/profesional para la gestión de un Sistema de Cine.